Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Dizilerin (statik ve dinamik) sınırlamaları
- Bağlı liste (linked list) veri yapısına giriş
- Bağlı listenin avantajları ve dezavantajları
- Tek bağlı liste (single linked list):
  - `struct` ile düğüm (node) tanımı ve `head` değişkeni
  - Düğüm oluşturma (`createNode`)
  - Başa ekleme (`insertAtFront`)
  - Sona ekleme (`insertAtEnd`)
  - Listeyi yazdırma (`printList`)
  - Araya (sıralı) ekleme mantığı
- İkili ağacı düz dizide tutmanın sorunu

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlı liste yapısı ikinci ve üçüncü sınıfta sürekli kullanılacak: ağaçlarda, graflarda, kuyruklarda. Bu yüzden düzgünce öğrenilmeli.
- `malloc` ile yer ayırdıktan sonra dönen göstericinin `NULL` olup olmadığı **mutlaka** kontrol edilmeli. "Nasılsa yer vardır" diye düşünülmemeli. Hocanın örneği: Uzaya füze gönderen bir programda milyonda bir ihtimalle yer sorunu çıkarsa füze yere çakılır.
- Her ekleme işleminde iki durum ayrı düşünülmeli: liste boşsa (`head == NULL`) ve listede eleman varsa.
- Özel bir gereksinim yoksa yeni elemanı listenin başına eklemek en verimlisidir. Sona eklemek için tüm listeyi gezmek gerekir.
- Araya ekleme yapabilmek için bir önceki düğümün adresi de (previous) bir değişkende tutulmalıdır.
- Global değişken kullanmaktan kaçınılmalı. Ama `head` gibi tüm fonksiyonların kullandığı bir değişkenin global tanımlanmasında sakınca yoktur.
- Bu derste amaç C dilini öğretmek değil, iyi algoritma tasarımını görmektir. Dilin yalnız `if`, `for`, `while` gibi temel yapıları kullanılır. Kütüphane ayrıntıları başka derslerin konusudur.

Kısa Tekrar Notları

- **Dizi:** Bellekte ardışık yerleşir. Her elemana adresiyle doğrudan erişilir (rastgele erişim, random access).
- **Statik dizi sorunu:** `int A[50]` gibi bir tanımda n'in 50'yi aşmaması garanti edilmelidir.
- **Dinamik dizi:** `malloc(n * sizeof(int))` heap bölgesinde n eleman için ardışık bir yer ayırır. Ama eleman sayısı hiç bilinmiyorsa veya sık ekleme/silme varsa yine sorun çıkar.
- **Bağlı liste:** Her eleman için ayrı yer ayrılır. Her düğüm bir değer (value) ve bir sonraki düğümün adresini (next) tutar. Son düğümün `next` alanı `NULL`'dır. `head` ilk düğümün adresini tutar.
- **Avantajlar:**
  - Boyut sabit değildir. Eleman eklendikçe büyür.
  - Eleman ekleme ve silme kolaydır.
  - Elemanların bellekte ardışık olması gerekmez.
- **Dezavantajlar:**

  - Rastgele erişim yoktur. Bir elemana ulaşmak için baştan itibaren sırayla gidilir.
  - Her düğüm için fazladan bir gösterici (pointer) alanı gerekir.
  - Düğümler bellekte dağınık durduğu için önbellek (cache) dostu değildir.
- **Ekleme türleri:** başa, sona, araya (ihtiyaca göre uygun yere).
- **Başa ekleme:** `newNode->next = head; head = newNode;` Liste boşken de doğru çalışır.
- **Sona ekleme:** Liste boşsa `head = newNode`. Değilse `tmp->next != NULL` olduğu sürece ilerlenir, sonra `tmp->next = newNode` yapılır.

Detaylı Açıklamalar

Dizilerden Bağlı Listeye

İlk haftalarda arama ve sıralama algoritmaları hep diziler üzerinde görüldü. Dizi bilinen en temel veri yapısıdır:

- Aynı amaç için kullanılan birden fazla bilgi, ayrı değişkenler yerine bellekte ardışık bir bölgede tutulur.
- Her elemanın kendi yeri vardır. Adres kullanılarak örneğin A dizisinin 3. gözüne doğrudan erişilir.

Statik dizi: `int A[50]` gibi bir yer ayrılır, dışarıdan okunan n kadarı kullanılır. Bu durumda n'in 50'yi aşmaması garanti edilmelidir (geçerli indisler 0-49'dur).

Dinamik bellek ayırma (dynamic memory allocation): Dizi uzunluğu bazen 50 bazen 1000 olabilir. En kötü duruma göre gereksiz büyük bir statik yer tutmak yerine n dışarıdan alınır ve o çalıştırma için gereken kadar yer ayrılır. `n * sizeof(int)` ile heap bölgesinde n elemanı saklayacak kadar **ardışık** bir bölge ayrılır.

Bazı durumlarda bu da yetmez:

- Kaç eleman olacağı baştan hiç bilinmez. Yeni elemanlar geldikçe onlar için yer tutmak gerekir.
- Ekleme ve silme çok sık yapılır. Bir eleman gelir, işi bitince gider.
- Silmek fiziksel bir silme değildir. "O yeri bıraktım, başkası kullansın" demektir.

İkili arama ağacı (binary search tree) örneği: Ayrık Matematik dersinde görülen ikili arama ağacına yeni gelen bir eleman uygun yere eklenir. İşi biten bir eleman ağaçtan çıkarılır. Başlangıçta ağacın ne kadar büyüyeceği bilinmez. Gerçek dünyada her şey baştan belli değildir, elemanlar geldikçe işlem yapılır. Bu tür yapılarda bağlı liste veri yapısı kullanılır.

Bağlı Liste Yapısı

- Her seferinde tek bir eleman için yer ayrılır.
- Elemanlar geldikçe eklendiği için bellekte arka arkaya boş yer bulunacağının garantisi yoktur. Bu yüzden elemanlar ardışık duramaz.
- Her düğümde iki kısım vardır:
  - **Değer (value):** Basitlik için tek bir tam sayı. Gerçekte birden fazla bilgi olabilir; örneğin ad, 1999 gibi bir sayı, bir numara.
  - **Adres (next):** Bir sonraki düğümün bellekteki adresi. Bu olmazsa düğümler birbirine tutunamaz, kaybolur.
- Son düğümün `next` alanı `NULL`'ı gösterir.
- Ayrıca ilk düğümün adresini tutan bir değişken gerekir: `head`. `head` değer tutmaz, yalnız işe nereden başlanacağını gösterir.

Örnek: Birinci düğüm 1000 numaralı adreste olsun. Yeni gelen düğüm 1400 numaralı adrese yerleşirse birinci düğümün `next` alanı 1400 olur. Sonra gelen düğüm 1500'e yerleşirse 1400'deki düğümün `next` alanı 1500 olur. Sonrasında düğüm yoksa 1500'deki `next` alanı `NULL` olur.

Doğrusal ve sıralı erişim: Bağlı liste de dizi gibi doğrusal (lineer) bir veri yapısıdır. Ama dizideki gibi bir elemana birden erişilemez. Kasetçalar mantığıyla sırayla ilerlemek gerekir:

- 9'a ulaşmak için önce `head`'den başlanır. `head` ilk düğüme götürür.
- İlk düğümün `next` alanına bakılır, 1400'e gidilir. 9 orada bulunur.

- İstenen elemana gitmek için önceki bütün elemanların üzerinden geçmek gerekir.

Avantajlar ve Dezavantajlar

Öğrencilerle birlikte çıkarılan liste:

Avantajlar:

- Boyut sabit değildir. Eleman eklendikçe dinamik olarak büyür.
- Yeni eleman eklemek ve aradan eleman silmek kolaydır.
- Elemanların bellekte ardışık olması gerekmez. Dinamik ayırmada yeterince büyük ardışık boş yer yoksa sorun çıkardı. Bellekte kopuk kopuk boş bölgeler olsa bile bağlı liste bunları kullanabilir.

Dezavantajlar:

- **Rastgele erişim (random access) yoktur.** En büyük sıkıntı budur. Örneğin elemanlar sıralı olsa bile ikili arama (binary search) avantaj sağlamaz. $n/2$. elemana gitmek için $n/2$ adım ilerlemek gerekir. Bu yüzden $\log n$ karmaşıklıklı bir arama yapılamaz.
- **Ek yer ihtiyacı:** Her düğüm için bir sonraki düğümü gösteren bir gösterici (pointer) değişken gerekir. 1000 eleman varsa 1000 tane de gösterici vardır.
- **Önbellek (cache) dostu değildir:** Sık kullanılan elemanlar önbellekte tutulur. Bir dizi bütün olarak önbelleğe alınabilir. Bağlı liste düğümleri ise belleğin farklı bölgelerinde durduğu için önbellekten iyi yararlanılamaz.

Bir öğrenci "dikkatli olmazsak erişimi kaybedebiliriz" dedi. Hoca bunun yapının değil, kullanıcının hatası olduğunu söyledi.

Sonuç: Sık ekleme ve silme gerekiyorsa, eleman sayısı sabit değilse ve ne zaman yeni eleman geleceği belli değilse bağlı liste diziden çok daha avantajlıdır. Ağaçlar (trees) ve graflar (graphs) bağlı liste yapısını yoğun biçimde kullanır.

Bağlı Liste Türleri

- **Tek bağlı liste (single linked list):** Her düğüm yalnız kendisinden sonra gelen düğümü gösterir. Bu derste önce bu anlatıldı.
- **Çift bağlı liste (double linked list):** Her düğüm hem kendisinden önceki hem sonraki düğümü gösterir. Daha sonra anlatılacak.

Düğüm Tanımı ve Düğüm Oluşturma

Kod (ekrandan, 00:35:05; `p->next = NULL;` satırını hoca ders içinde sonradan ekledi):

```c
#include <stdio.h>    /* printf için gerekli; tahtada yoktu */
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} NODE;

NODE *head;

NODE *createNode() {
    NODE *p;
    p = (NODE *) malloc(sizeof(NODE));
    if (p == NULL) {
        printf("Allocation error\n");
        exit(0);
    }
    p->next = NULL;
    return p;
}
```

- Düğüm hem değeri hem bir sonraki düğümün adresini saklamak zorunda olduğu için `struct` kullanılır.
- `next` alanının tipi yine `struct Node *`'dır. Bir sonraki eleman da aynı tipte bir düğüm olduğu için burada özyinelemeli (recursive) bir tanım vardır.
- `head` global tanımlandı. Normalde global değişken kullanmak kötüdür:
  - Fonksiyondan fonksiyona geçerken değişkenin durumunu takip etmek zorlaşır.
  - Değişkeni hangi fonksiyonların kullandığını izlemek zordur.
  - Değişken gereksiz yere tutulur.
- Ama `head` tüm fonksiyonların kullandığı bir değişkendir. Bu yüzden global tanımlanmasında sakınca yoktur.
- `createNode` fonksiyonu yeni düğümün adresini döndürdüğü için dönüş tipi `NODE *`'dır. `malloc`'un döndürdüğü adres `(NODE *)` ile tip dönüşümüne (casting) uğratılır.
- Yer bulunamazsa örnekte `exit` ile çıkıldı. Gerçek bir programda algoritmanın ihtiyacına göre ne yapılması gerekiyorsa o yapılır.
- Hoca, `next = NULL` atamasını her ekleme algoritmasında ayrı ayrı yapmak yerine düğüm oluşturulurken bir kez yapmanın daha şık olduğunu söyledi.

Kullanım örneği (ekrandan):

```
int main()
{
    NODE *p;
    head = NULL;
    p = createNode();
    insertAtFront(p, 8);
    ...
}
```

Bir öğrenci, `createNode` fonksiyonunu `main` yerine ekleme fonksiyonlarının içinden çağırmanın daha az parametre gerektireceğini sordu. Hoca bunun doğru olduğunu söyledi:

- Ekleme yapmadan önce düğümün oluşturulması gerekir. Nerede oluşturulacağı ihtiyaca bağlıdır.
- Yer veya zaman açısından fark yoktur.
- Hoca, `create` ve `insert` işlemleri bağımsız olarak görülsün diye onları ayrı yazdı. Gerçek programlarda `main` başka işler yapıyor olabilir, bağlı liste yalnız bir fonksiyonun içinde kullanılıyor olabilir.

Ekleme (Insert) Türleri

Yeni elemanın nereye ekleneceği algoritmanın ihtiyacına bağlıdır:

1. **Başa ekleme:** Özel bir durum yoksa en verimli yoldur. Yeni eleman için n adım ilerleyip sona gitmeye gerek kalmaz. Bunu bir öğrenci önerdi: "Sona kadar gitmek yerine tek bir değişkeni değiştirmek daha kolay olmaz mı?"
2. **Sona ekleme:** Algoritma bunu gerektiriyorsa yapılır. Örneğin çok kullanılan elemanlar öndeyse ve yeni gelen eleman ilk kez kullanılıyorsa sona konabilir.
3. **Araya ekleme:** Elemanlar örneğin sıralı tutuluyorsa yeni eleman uygun yere konmalıdır. İkili arama ağacında da yeni eleman istenen yere konamaz. Örneğin 7 geldiyse 5'ten büyük, 9'dan küçük olduğu için ikisinin arasına yerleşir.

Başa Ekleme (Insert at Front)

Kod (ekrandan, 00:45:15):

```
void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}
```

Eleman varsa: `head` 1000'i gösteriyor, 1000'deki düğümde 9 var. Yeni düğüm 1500'de ve değeri 8.

- `newNode->next = head` ile yeni düğüm 1000'i gösterir.
- `head = newNode` ile `head` artık 1500'ü gösterir.
- `head` ile eski ilk düğüm arasındaki bağ koparılmış, yeni düğüm araya girmiş olur.

Eleman yoksa (`head == NULL`):

- `newNode->next = head` ile yeni düğümün `next` alanı `NULL` olur.
- `head = newNode` ile `head` 1500'ü gösterir.
- Yani algoritma boş liste için de ayrı bir kontrol gerektirmeden doğru çalışır.

Sona Ekleme (Insert at End)

Kod (ekrandan, 01:05:25):

```
void insertAtEnd(NODE *newNode, int value)
{
    NODE *tmp;
    newNode->value = value;   /* tahtada yoktu; bkz. ders sonrası notlar */
    if (head == NULL)
        head = newNode;
    else
    {
        tmp = head;
        while (tmp->next != NULL)
            tmp = tmp->next;
        tmp->next = newNode;
    }
}
```

- Başa eklemeden farklı olarak burada listenin boş olup olmadığı **kontrol edilmek zorundadır**. Kontrol edilmezse `NULL`'ın arkasına eklenmeye çalışılır ve hata oluşur.
- Liste boşsa `head` doğrudan yeni düğümü gösterir.
- Eleman varsa sona kadar gidilir. Sona gitmek, `next` alanı `NULL`'dan farklı olduğu sürece ilerlemektir.
- Listede ilerlemek için geçici bir gösterici (`tmp`) kullanılır; böylece `head` değişmez.

Örnek: `head` → 1000 (8) → 1500 (9) → `NULL`. Yeni düğüm 1200'de, değeri 4.

Adım	`tmp`	`tmp->next`	Karar
1	1000	1500	`NULL` değil, ilerle
2	1500	`NULL`	Döngüden çık
Sonuç			`tmp->next = 1200`: 1500'deki düğüm artık 1200'ü gösterir

Listeyi Yazdırma

Kod (ekrandan, 01:12:40):

```
void printList()
{
    NODE *ptr;
    ptr = head;
    while (ptr != NULL)
    {
        printf("%d\n", ptr->value);
        ptr = ptr->next;
    }
}
```

Örnek: `head` → 1000 (5) → 1200 (9) → 1500 (7) → `NULL`.

- `ptr` 1000 iken 5 yazılır. `ptr = ptr->next` ile 1200 olur.
- 9 yazılır, `ptr` 1500 olur.
- 7 yazılır, `ptr` `NULL` olur. Döngüden çıkılır.
- Sonuçta 5, 9 ve 7 ekrana yazılmış olur.

İkili Ağacı Düz Dizide Tutmanın Sorunu

Hoca araya eklemeye geçmeden önce ağaç yapısında neden düz dizi kullanılmaması gerektiğini gösterdi (ekrandan, 01:20:20).

Ağaç: Kök 9. 9'un solunda 7, sağında 12. 7'nin solunda 5. 5'in solunda 3. 3'ün sağında 4.

Düz dizide i. gözdeki düğümün çocukları $2i + 1$ ve $2i + 2$. gözlere yerleşir:

```
indeks : 0  1   2  3  4  5  6  7  ...  16
değer  : 9  7  12  5  -  -  -  3  ...   4
```

- 3, 7. göze yerleşir. Aradaki 4, 5 ve 6. gözler boş kalır.
- 3'ün sağ çocuğu 4, $2 \cdot 7 + 2 = 16$. göze yerleşir. Arası bomboştur.
- Ağaç yapısında düz dizi kullanmanın en büyük sıkıntısı budur: çok fazla boş yer kalır.

Araya Ekleme (Sıralı Liste)

Hoca araya eklemenin mantığını anlattı. Kodunu derste yazmadı, öğrencilerin bilgisayar başında kendilerinin yazmasını istedi. Hocaya göre bir şeyi yapmadan yalnız dinleyerek iyice öğrenmek mümkün değildir.

Yine iki durum vardır:

1. **Eleman yoksa:** `head` doğrudan yeni düğümü (örneğin 1200) gösterir.
2. **Eleman varsa:** Elemanların küçükten büyüğe sıralı tutulduğunu varsayalım. (Araya eklemenin tek senaryosu bu değildir, her türlü durum olabilir.)

Örnek (ekrandan): `head` → 1200 (3) → 1500 (4) → 1700 (9) → `NULL`. Yeni düğüm 1800'de, değeri 7.

- 7, 3 ile karşılaştırılır. Büyük, 1500'e gidilir.
- 7, 4 ile karşılaştırılır. Büyük, 1700'e gidilir.
- 7, 9 ile karşılaştırılır. Küçük, o hâlde 9'dan önce gelmelidir.
- Artık 4'ün `next` alanı 1700 değil 1800 olmalı. 7'nin `next` alanı da 1700 olmalı.

Anahtar nokta: 4'ün `next` alanını değiştirebilmek için 4'ün düğümünü bilmek gerekir. Yalnız `next` adresini izlemek yetmez. Her ilerlemede bir önceki düğümün adresi (previous) de bir değişkende tutulmalıdır.

Bir öğrenci listenin sonuna gelinip gelinmediğinin de kontrol edilmesi gerektiğini sordu. Hoca onayladı: Örneğin 15 gelseydi, liste bittiği noktada 15 en sona yerleştirilmelidir.

Hocanın önerdiği alıştırma: Elemanları küçükten büyüğe sıralı bir bağlı liste oluşturun. Verilen bir elemanı sırayı bozmayacak şekilde listeye yerleştirin. Liste, her gelen eleman döngü içinde sıralı yerleştirilerek de oluşturulabilir.

Örnek çözüm (derste yazılmadı; ajan tarafından eklendi, derlenip `4 9 3 7 15 1` girdisiyle test edildi):

```c
void insertSorted(NODE *newNode, int value)
{
    NODE *prev = NULL, *cur = head;
    newNode->value = value;
    while (cur != NULL && cur->value < value) {
        prev = cur;
        cur = cur->next;
    }
    newNode->next = cur;
    if (prev == NULL)     /* liste boş ya da yeni eleman en küçük */
```

```
        head = newNode;
    else
        prev->next = newNode;
}
```

Ders, tek bağlı listeden düğüm silme konusuna geçilirken sona erdi.

Ders sonrası notlar

- Tahtadaki `insertAtEnd` kodunda `newNode->value = value;` satırı yoktu. O hâliyle `value` parametresi hiç kullanılmaz ve yeni düğümün değeri atanmamış kalır. Yukarıdaki kodda bu satır eklendi.
- `createNode` içinde `printf` kullanıldığı için `#include <stdio.h>` de gerekir. Tahtada yalnız `stdlib.h` yazılıydı; yukarıda eklendi.
- Araya eklemede hoca elde tutulacak adresi "1700" diye söyler (01:19:46). Doğrusu 1500'dür: değiştirilecek olan, 4'ün düğümünün `next` alanıdır ve bu düğümün adresi 1500'dür. 1700 ise 7'nin `next` alanına yazılacak adrestir.
- `exit(0)` normalde başarılı çıkışı belirtir. Hata durumunda `exit(1)` veya `exit(EXIT_FAILURE)` kullanmak daha doğrudur.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.